

Discovery Executive Summary

Project: discovery-123 · Generated: 15/07/2026, 20:15:33

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 6 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	44 / 100 — Moderate
3	Frontend Modernization Analysis	HIGH RISK	—
4	Backend Modernization Analysis	HIGH RISK	—
5	Testing & Quality Assurance Analysis	HIGH RISK	—
6	Security Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk F5 (legacy/inconsistent component patterns) and H10 (duplicate user-fetch coupling across 11 components).

EXECUTIVE SUMMARY

TARGET_WORKSPACE is a **frontend-only** discovery scope: the primary application is `social-media-react` (React 18, Redux, React Router v5, Axios) with **88** JavaScript/JSX source files across **57** page/component modules; a secondary CRA scaffold `workbench-demo` contributes **8** source files. **No server-side controllers, models, or repositories exist in this workspace** — persistence is delegated to an external Express API at `localhost:3030/api/`. Architecture is **Moderate-to-High Risk** on the frontend: a partial service layer (`src/services/`) is routinely bypassed by **16** components that import services directly, **11** components each re-fetch users via `userService.getById` (N+1 pattern), and **11** files use inconsistent `export default` vs named-

export conventions. Backend hotspots H1–H9 are **Not observed** (no server layer present). Overall rating is **High Risk**, driven by **F5** (legacy/inconsistent component patterns) and **H10** (duplicate cross-component data-fetch coupling).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	N/A (0 controllers)	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	N/A (0 controllers)	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 detected	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	0	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	N/A (no SQL layer)	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	N/A (no backend domains)	GOOD
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	N/A (external API)	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	80 avg (max 250)	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	16	MODERATE
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3 levels	MODERATE
F5	Legacy / Inconsistent	Legacy-pattern components	0	1–10	>10	11 default-export files	HIGH RISK

	Component Patterns						
H10	Duplicate User-Fetch Coupling (additional)	<code>userService.getById</code> call sites in UI	<5	5–10	>10	11	HIGH RISK

1.4 Actions Required

Hotspot	Action	Rating	Priority
H10	Add <code>usersById</code> entity map and <code>ensureUser</code> thunk; replace 11 component-level <code>userService.getById</code> calls with selector + deduplicated fetch	HIGH RISK	CRITICAL
F5	Standardize named exports across 11 default-export files; add <code>ErrorBoundary</code> ; plan React Router v6 migration	HIGH RISK	HIGH
F2	Route all component data access through Redux thunks; add ESLint rule banning <code>services/</code> imports in <code>cmps/</code> and <code>pages/</code>	MODERATE	HIGH
F4	Introduce <code>useLoggedInUser</code> context; pass composite <code>post</code> object to children; reduce 3-level prop chains	MODERATE	MEDIUM

1.5 Expected Outcomes

- Eliminating N+1 user fetches will cut API traffic on feed/notifications pages by an order of magnitude and remove per-card loading flicker.
- Enforcing thunk-only data access creates a single place for loading states, error handling, and socket invalidation.
- Extracting connection and reaction workflows into shared services removes duplicated business rules across `Profile` , `PostPreview` , and `CommentPreview` .
- Standardizing exports and migrating to React Router v6 reduces onboarding friction and enables modern data-router patterns.
- Redux Toolkit slices with normalized entities establish bounded contexts that prepare the app for feature-level code splitting or micro-frontend extraction.

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by High-Risk cyclomatic complexity in frontend page components (H1), oversized ConnectPage function (H3), cross-runtime business-logic duplication (H4), parallel Laravel/dev-api workflow copies (H9), and PHP `extract()` dynamic variables (H11).

EXECUTIVE SUMMARY

Analysis covered **backend** (16 PHP application files, 807 SLOC), **frontend** (14 TS/TSX/JSX files, 874 SLOC), and **dev-api** (6 JS files, 720 SLOC) — **36 files / 2,401 SLOC** total — from `shende-shweta/FSDKC@main` via shallow git clone and manual inspection (GitHub API rate-limited during run; no cyclomatic-complexity linter configured). The dominant risks are **frontend page-level complexity** (`ConnectPage.tsx` cyclomatic complexity **37**, **201 LOC** default-export function), **cross-runtime business-logic duplication** (~**13.2%** of codebase duplicated between Laravel and `dev-api` for KPI aggregation, realtime test orchestration, and IVR `buildTree`), and **three `extract()` dynamic-variable sites** in legacy PHP code. No files exceed 1,000 LOC (largest: `dev-api/src/server.js` at **224 SLOC**). Git history is shallow (**3 commits**, single author `ksabai-gl` since June 2026), so churn and ownership signals are healthy but low-confidence. Overall verdict: **High Risk**, driven by H1, H3, H4, H9, and H11.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	37 (<code>ConnectPage</code>)	HIGH RISK
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	224 LOC (<code>dev-api/src/server.js</code>)	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	201 LOC (<code>ConnectPage</code>)	HIGH RISK
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	13.2% (317 / 2,401 LOC)	HIGH RISK
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	6.6% (106 / 1,228 sig. lines)	MODERATE
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	2 changes/mo (top app files)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1 fix commit (<code>frontend/src/App.tsx</code>)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (1 author / 3 commits)	GOOD
H9	Parallel Runtime Duplication (additional)	Duplicated workflow LOC across Laravel + dev-api / backend LOC	<5%	5–15%	>15%	20.8% (317 / 1,527 backend+dev-api LOC)	HIGH RISK
H10	God Page Components	Largest page component	<150	150–300	>300	208 LOC (<code>ConnectPage.tsx</code>)	MODERATE

	(additional)	LOC (UI + data + realtime)					
H11	PHP extract() Dynamic Variables (additional)	<code>extract()</code> call sites in application code	0	1–2	>2	3 (<code>LegacyReportController</code> , <code>LegacyDataMapper</code> ×2)	HIGH R

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	74	18.5
Code Churn	25%	12	3.0
Defect Density	20%	15	3.0
Class/Function Size	15%	75	11.25
Business Logic Duplication	10%	78	7.8
Developer Ownership Risk	5%	5	0.25
Hotspot Score	100%		44 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — High Cyclomatic Complexity	Extract <code>useModulePageQueries</code> hook; split <code>ConnectPage</code> and <code>DiscoveryPage</code> into ≤80 LOC sub-components; enable ESLint complexity rule (max 15) on <code>frontend/src/pages/</code> .	HIGH RISK	CRITICAL
H3 — Large Functions	Decompose <code>ConnectPage</code> (201 LOC) into <code>useConnectPageState.ts</code> + 3 presentational components; apply same pattern to <code>DiscoveryPage</code> (154 LOC).	HIGH RISK	HIGH
H4 — Business Logic Duplication	Create <code>DashboardKpiCalculator</code> and shared test-step definitions in Laravel; remove duplicated KPI and realtime blocks from <code>dev-api/src/server.js</code> and <code>dev-api/src/realtime.js</code> .	HIGH RISK	CRITICAL
H5 — Duplicate Code	Extract <code>TreeBuilder</code> utility from four <code>buildTree</code> copies; add <code>jscpd</code> / <code>phpcpd</code> CI gate at 5% threshold.	MODERATE	MEDIUM
H9 — Parallel Runtime Duplication	Deprecate <code>dev-api</code> runtime or auto-generate from Laravel OpenAPI; add contract tests until removal.	HIGH RISK	CRITICAL
H10 — God Page Components	Introduce <code>ModulePageLayout</code> , <code>EntityForm</code> , and <code>TestSessionPanel</code> shared components; cap page files at 120 LOC.	MODERATE	MEDIUM
H11 — PHP extract() Dynamic Variables	Replace <code>extract(\$filters)</code> with typed <code>CarrierSummaryFilter</code> DTO; refactor <code>LegacyDataMapper</code> to explicit key access; add PHPStan <code>extract()</code> ban.	HIGH RISK	HIGH

2.6 Expected Outcomes

- **Lower defect rate on UI changes:** Splitting `ConnectPage` / `DiscoveryPage` and extracting shared hooks reduces cyclomatic complexity from 37 to a testable target of <15 per function.
- **Single source of truth for KPIs and test pipelines:** Consolidating dashboard and realtime logic in Laravel eliminates the ~13% duplicated business-rule surface between production and `dev-api`.
- **Safer refactors:** Extracting `TreeBuilder` and domain services lets IVR tree and reachability changes propagate from one module instead of four `buildTree` copies.
- **Eliminated dynamic-variable risk:** Replacing `extract()` with typed DTOs enables PHPStan static analysis and prevents variable-injection bugs in legacy report endpoints.
- **Faster reviews:** Smaller page components and a complexity lint gate keep new features from re-expanding god components.

Full report saved to `target/docs/discovery/02-code-quality-complexity.md` (568 lines, including §2.2 evidence, §2.3 churn tables, and §2.4 Mermaid diagrams). Pipeline summary: `agent-runs/20260715T200939_cg7j6r/02-code-quality-complexity-summary.md`.

3. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

Driven by UI component duplication (H1), prop-drilling depth of 6 (H5), and 0% Redux Toolkit adoption (H6).

EXECUTIVE SUMMARY

The target workspace contains two Create React App frontends; the primary application is `social-media-react` (59 of 61 scanned view files), a functional-component React 18 app with legacy Redux (`createStore` + thunks, no Redux Toolkit). All scanned components use hooks—no class components were found. The largest risks are duplicated preview/reaction UI across eight components (~13%), prop-drilling chains up to six levels in the post/comment/reply tree, and pervasive global Redux reads (46% of view files). The largest single file is `Message.jsx` at 250 LOC (moderate, below the 500 LOC threshold). `workbench-demo` is a small, modern TypeScript login shell with no shared component library linkage to the main app.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	13.1% (8/61)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	100% (61/61)	GOOD
H3	Massive Components	Largest component LOC	<200	200–500	>500	250 (<code>Message.jsx</code>)	MODERATE
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	45.9% (28/61)	MODERATE
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	6 levels	HIGH RISK
H6	Legacy Redux Without Toolkit (additional)	RTK slice/store adoption %	>90%	70–90%	<70%	0% (0/4 modules)	HIGH RISK

3.5 Actions Required

Hotspot	Action	Rating	Priority
H1 UI Component Duplication	Extract <code>useReactionToggle</code> , <code>UserAvatarCard</code> , and shared preview list row components; refactor eight duplicate files to consume them.	HIGH RISK	HIGH
H3 Massive Components	Extract <code>useChat</code> from <code>Message.jsx</code> ; split <code>CreatePostModal.jsx</code> and <code>Signup.jsx</code> into hooks + presentational subcomponents; enforce 200 LOC soft limit.	MODERATE	MEDIUM
H4 Global State Dependencies	Migrate to Redux Toolkit slices; scope session data via Context; remove <code>window.myBus</code> global export.	MODERATE	MEDIUM
H5 Complex State Management	Add <code>PostInteractionContext</code> at post boundary; replace six-level callback drilling with context or RTK entity dispatches.	HIGH RISK	HIGH
H6 Legacy Redux Without Toolkit	Add <code>@reduxjs/toolkit</code> , convert four modules to slices, relocate socket listeners to listener middleware.	HIGH RISK	CRITICAL

3.6 Expected Outcomes

- A shared component library (`UserAvatarCard` , `ReactionToggle` , preview rows) reduces duplicated UI logic from ~13% toward the <5% target and stabilizes like/avatar behavior across feed, comments, and messages.
- Extracted hooks (`useChat` , `useReactionToggle`) and sub-200 LOC page components improve unit-test coverage and enable incremental TypeScript adoption.
- Redux Toolkit slices with colocated selectors cut boilerplate action/reducer code by roughly half and make socket-driven updates traceable via listener middleware instead of `Main.jsx` wiring.

- Context and entity-based dispatches collapse prop-drilling from six levels to ≤ 3 , simplifying future feature work and cross-app patterns for `workbench-demo` integration.

4. Backend Modernization Analysis

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

High Risk

Driven by Direct SQL/ORM Outside Data Layer (H3), Missing Service Layer (H5), API Sprawl (H6), Missing API Governance (H7), and Parallel API Runtimes (H8).

EXECUTIVE SUMMARY

Analysis covered **24 controllers/handlers** (6 Laravel API controllers + 18 Express route handlers), **37 REST endpoints** across two parallel runtimes, and **2 injectable service classes** with **0 repository classes**, sourced from `shende-shweta/FSDKC@main` via GitHub REST API (public tree + raw content fetch). The Klearcom backend runs Laravel 12 with constructor-injected `MongoService` and `RealTimeTestService`, but **25 of 35 Eloquent access points (71%)** remain in controllers with no repository layer, and four controllers embed KPI math, IVR tree building, and reachability calculations inline. Three PHP `extract()` calls — including one on raw `$request->all()` — create untyped variable scope from user input. The Node `dev-api` holds all relational state in a module-level mutable `store` singleton with **18 inline route handlers** mirroring Laravel. No OpenAPI spec, API versioning, or contract tests exist; CI runs PHPUnit and frontend build only. Overall verdict: **High Risk**, driven by data-layer bypass (H3), missing service tier across dual runtimes (H5), API sprawl and zero governance (H6–H7), and parallel API implementations (H8).

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3 (<code>extract()</code> calls)	MODERATE
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	2 module-level stores	MODERATE
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	29% (10/35 Eloquent in services; 0 repositories)	HIGH RISK
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	0	GOOD

H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	22 (4 Laravel controllers + 18 dev-api routes)	HIGH RISK
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	5% single-source (1/19 capabilities); 0% governed	HIGH RISK
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% (no spec, versioning, or contract tests)	HIGH RISK
H8	Parallel API Runtimes (additional)	Capabilities with duplicate Laravel + dev-api handlers (target 0)	0	1–5	>5	17 duplicated capabilities	HIGH RISK
H9	Missing Input Validation on dev-api (additional)	POST/PUT routes without schema validation (target 0)	0	1–3	>3	4 unvalidated write routes	HIGH RISK

4.5 Actions Required

Hotspot	Action	Rating	Priority
H1 — Dynamic Variable Creation	Replace <code>extract(\$filters)</code> in <code>LegacyReportController</code> with validated request DTO; refactor <code>LegacyDataMapper</code> to explicit field access; add CI ban on <code>extract()</code> .	MODERATE	HIGH
H2 — Global Mutable State	Encapsulate <code>dev-api/src/store.js</code> in an injectable repository; eliminate module-level business mutation or retire dev-api.	MODERATE	MEDIUM
H3 — Direct SQL Outside Data Layer	Create Eloquent repositories for all four models; move 25 controller ORM calls into repositories; target >90% data-layer compliance.	HIGH RISK	CRITICAL
H5 — Missing Service Layer	Add <code>DashboardService</code> , <code>DiscoveryService</code> , <code>ConnectService</code> ; move KPI/tree/reachability logic out of controllers and dev-api routes.	HIGH RISK	CRITICAL
H6 — API Sprawl	Consolidate 17 duplicated capabilities under Laravel; deprecate overlapping dev-api routes; publish canonical endpoint list.	HIGH RISK	HIGH
H7 — Missing API Governance	Author <code>docs/openapi.yaml</code> ; add <code>/api/v1/</code> versioning; introduce contract tests in CI via schemathesis or equivalent.	HIGH RISK	HIGH
H8 — Parallel API Runtimes	Retire or proxy <code>dev-api</code> to Laravel; eliminate triplicated reachability and dashboard KPI logic.	HIGH RISK	CRITICAL
H9 — Missing Input Validation on dev-api	Add express-validator schemas on all POST routes; remove or secure <code>bulk-import</code> endpoint.	HIGH RISK	HIGH

4.6 Expected Outcomes

- **Repository layer** moves 25 controller Eloquent calls behind testable boundaries, raising data-layer compliance from 29% to >90% and enabling mocked persistence in PHPUnit feature tests.
- **Application services** (`DashboardService` , `DiscoveryService` , `ConnectService`) eliminate triplicated KPI, tree, and reachability logic — fixes ship once and propagate to all entry points.
- **Retiring parallel dev-api routes** removes 17 duplicate handlers and the module-level `store` singleton, halving API surface area and integration drift risk.
- **OpenAPI spec + contract tests** catch breaking response-shape changes before merge, giving the React SPA a machine-verifiable integration contract.
- **Replacing `extract()` with typed DTOs** closes the variable-scope injection vector in legacy reporting and aligns with `AGENTS.md` engineering standards.

Full report saved to `target/docs/discovery/04-backend-modernization.md` . Pipeline summary saved to `agent-runs/20260715T200939_cg7j6r/04-backend-modernization-summary.md` .

5. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by H1 (7 untested critical modules), H2 (~6% overall coverage), H3 (0% integration boundaries), H4 (0% contract tests), H6 (no CI gate), and H7 (no E2E tests).

EXECUTIVE SUMMARY

The target workspace is a **frontend-only** monorepo containing two Create React App projects: `workbench-demo` (TypeScript login demo) and `social-media-react` (Redux social-network client). **Backend tests:** not applicable — no server-side source (PHP, Python, Java, etc.) is present under the target root. **Frontend tests:** `workbench-demo` has a focused Jest suite (16 passing tests) with **86% measured statement coverage** on its small surface area, but `App.tsx` and `index.tsx` remain untested. `social-media-react` has **~83 source modules** and **only one stale CRA boilerplate test** (`App.test.js` asserts "learn react", which the app no longer renders); its test runner could not execute in this environment because `node_modules` is not installed. **Overall estimated coverage across both apps is ~6%** (file-weighted). Critical auth, routing, HTTP, and Redux logic in `social-media-react` ships with **zero automated tests**. There are **no integration, contract, or E2E tests**, **no coverage thresholds**, and **no CI workflow** under the target workspace that runs tests on change.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	7 modules	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~6% overall (86% workbench-demo measured; ~0% social-media-react estimated)	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (all HTTP/auth tests use mocks)	HIGH RISK
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (REST endpoints consumed but not contract-tested)	HIGH RISK
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	GOOD
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	No <code>.github/workflows</code> under target	HIGH RISK
H7	No End-to-End Tests (additional)	Critical user journeys with E2E specs	>70%	30–70%	<30%	0 journeys (no Cypress/Playwright)	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Untested Critical Logic	Add Jest tests for <code>userService</code> , <code>PrivateRoute</code> , <code>httpService</code> , <code>userActions</code> , and <code>Signup</code> in <code>social-media-react</code> before any refactor	HIGH RISK	CRITICAL
H2 Low Test Coverage	Establish baseline coverage in <code>social-media-react</code> ; raise overall workspace coverage from ~6% toward 75% on services/actions first	HIGH RISK	CRITICAL
H3 Missing Integration Tests	Introduce MSW-based integration tests for login/signup Redux chains; reduce pure module mocking in <code>workbench-demo</code>	HIGH RISK	HIGH
H4 Missing Contract Tests	Create JSON Schema fixtures and validation tests for auth and user API endpoints consumed by both apps	HIGH RISK	HIGH
H6 No CI Test Gate	Add GitHub Actions workflow running <code>CI=true npm test -- --watchAll=false --coverage</code> for both apps on every PR	HIGH RISK	HIGH
H7 No End-to-End Tests	Add Playwright smoke specs for login, signup-to-feed, and logout redirect in <code>social-media-react</code>	HIGH RISK	MEDIUM

5.5 Expected Outcomes

- Critical auth, route-guard, and HTTP modules in `social-media-react` are protected by unit tests before any modernization or extraction work begins.

- MSW integration and JSON Schema contract tests catch API breaking changes before they reach the browser.
- A CI test gate on every pull request prevents merges that regress the existing 16-test `workbench-demo` suite or newly added `social-media-react` coverage.
- Playwright E2E smoke tests verify end-to-end login and signup journeys that unit tests alone cannot cover.
- Per-app coverage metrics (86% workbench-demo maintained; social-media-react raised from ~0%) give an honest picture of workspace test health.

Full report saved to `target/docs/discovery/05-testing-and-quality-assurance.md` (266 lines).
Pipeline artifact: `agent-runs/20260715T200939_cg7j6r/05-testing-and-quality-assurance-summary.md`.

6. Security Analysis

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by Critical missing API authentication (H1), High permissive CORS and session stream IDOR (H2), and 9/12 OWASP categories with concrete findings (H5).

EXECUTIVE SUMMARY

Security review covered **backend** (18 PHP application files, 6 API controllers, 19 public Laravel routes), **frontend** (15 TS/TSX/JSX source files), and **dev-api** (6 JS files, 18 Express routes) fetched from `shende-shweta/FSDKC@main` via GitHub REST API and raw content scan. The stack uses Eloquent ORM and the MongoDB PHP driver with parameterized queries — no SQL, NoSQL, or shell injection hotspots were observed. The dominant risk is **complete absence of authentication and authorization** on every API route (Laravel and dev-api), compounded by **wildcard CORS** (`allowed_origins: ['*']` / `cors()` with no origin filter), enabling any website or anonymous client to create jobs, start tests, read transcripts, and import monitors. A **session stream IDOR** lets any caller replay SSE events with only a `session_id` (the route `{id}` is ignored). PHP `extract($request->all())` in `LegacyReportController` creates variable-scope injection from query parameters. Frontend XSS sinks, client-side secrets, and browser token storage were not observed; `npm audit` reported **zero** CVEs across frontend and dev-api lockfiles. CI runs PHPUnit and frontend build only — no `npm audit`, `composer audit`, or SAST gate. Overall verdict: **High Risk**, driven by one Critical finding (missing API authentication), three High findings (permissive CORS, session IDOR, unauthenticated bulk-import), and 9/12 OWASP categories with concrete findings.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	--------------	--------	------	----------	-----------	----------	--------

H1	Critical Vulnerabilities	0	0	1	>1	1 (unauthenticated API surface)	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	3 (CORS wildcard, session IDOR, bulk-import)	GOOD
H3	Medium Vulnerabilities	low	<20	20–50	>50	7 (default creds, debug mode, missing headers, no rate limit, exposed DB ports, extract(), FS5 HTTP/CSP)	MODERATE
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	3.9/KLOC (11 findings / 2.8 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	25% categories clean (3/12)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	0 (<code>npm audit</code> critical+high on frontend and dev-api)	GOOD
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	0% flagged (Laravel 12, React 19, Vite 6 — current majors)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	0 EOL majors detected	GOOD

6.5 Actions Required

Finding	Action	Rating	Priority
Missing API Authentication	Add Sanctum/JWT auth middleware to all routes in <code>backend/routes/api.php</code> ; add policy checks in 6 controllers; mirror or remove unauthenticated dev-api.	HIGH RISK	CRITICAL
Permissive CORS	Replace <code>allowed_origins: ['*']</code> in <code>backend/config/cors.php</code> and default <code>cors()</code> in <code>dev-api/src/server.js</code> with env-driven allow-list.	HIGH RISK	HIGH
Session Stream IDOR	Bind <code>session_id</code> to <code>resource_id</code> + user in <code>RealTimeTestService</code> ; validate in <code>StreamController</code> and dev-api <code>streamSession()</code> .	HIGH RISK	HIGH
Unauthenticated Bulk Import	Remove or protect <code>dev-api /bulk-import</code> with auth, schema validation, and rate limiting.	HIGH RISK	HIGH
PHP extract() Variable Injection	Replace <code>extract(\$filters)</code> in <code>LegacyReportController</code> with typed DTO; refactor <code>LegacyDataMapper</code> to explicit key access.	MODERATE	MEDIUM
Default Credentials & Debug Mode	Remove hardcoded passwords from <code>docker-compose.yml</code> and <code>.env.example</code> ; set <code>APP_DEBUG=false</code> outside local.	MODERATE	MEDIUM

Missing Security Headers	Add CSP, HSTS, X-Frame-Options, and X-Content-Type-Options in <code>docker/nginx/default.conf</code> .	MODERATE	MEDIUM
No Rate Limiting	Apply Laravel <code>throttle</code> middleware and <code>express-rate-limit</code> on POST/start endpoints.	MODERATE	MEDIUM
Exposed Database Ports	Remove <code>3306:3306</code> and <code>27017:27017</code> host mappings from <code>docker-compose.yml</code> .	MODERATE	MEDIUM
FS5 — HTTP default & no CSP	Enforce HTTPS <code>VITE_API_URL</code> in production; add CSP to <code>frontend/index.html</code> or nginx.	MODERATE	MEDIUM
DevSecOps — no dependency scan in CI	Add <code>npm audit --audit-level=high</code> and <code>composer audit</code> steps to <code>.github/workflows/ci.yml</code> .	MODERATE	MEDIUM
No Security Audit Logging	Add structured audit log for resource create/start/stream events in Laravel.	MODERATE	MEDIUM

Full report saved to `target/docs/discovery/06-security.md` (478 lines). Pipeline summary: `agent-runs/20260715T200939_cg7j6r/06-security-summary.md` .